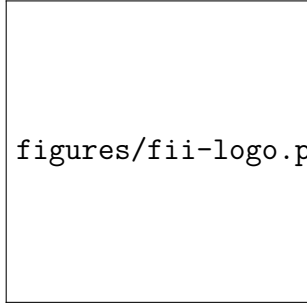


**“ALEXANDRU IOAN CUZA” UNIVERSITY OF IAȘI**  
**FACULTY OF COMPUTER SCIENCE**



figures/fii-logo.png

**BACHELOR'S THESIS**

**Sequire: A Real-Time Simulation and  
Visualization Platform for the BB84  
Quantum Key Distribution Protocol**

submitted by

**Vezeteu Andrei**

**Session:** July, 2026

*Scientific coordinator*  
**Conf. Dr. Andreea Arusoaie**

# Abstract

*[Abstract to be written after the main chapters are finalised, so the summary accurately reflects the final content. Expected length: one page. Topics to cover: the problem (need for accessible QKD experimentation), the proposed solution (Sequire platform), key technical contributions (real IBM Quantum hardware integration, real-time Web-Socket visualization, full end-to-end BB84 pipeline with EC and PA, multi-mode Eve simulation), methodology, and main results.]*

*Keywords:* BB84, Quantum Key Distribution, QKD, Quantum Cryptography, Qiskit, IBM Quantum, FastAPI, React, WebSockets, QBER, Privacy Amplification, Error Correction, No-Cloning Theorem.

# Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>3</b>  |
| 1.1      | Motivation . . . . .                                                | 3         |
| 1.2      | Proposed Solution . . . . .                                         | 4         |
| 1.3      | Contributions and Impact . . . . .                                  | 6         |
| <b>2</b> | <b>Theoretical Background</b>                                       | <b>9</b>  |
| 2.1      | Classical vs. Quantum Cryptography . . . . .                        | 9         |
| 2.2      | Quantum Mechanics Primer . . . . .                                  | 10        |
| 2.2.1    | Qubits and Quantum States . . . . .                                 | 10        |
| 2.2.2    | Measurement and State Collapse . . . . .                            | 10        |
| 2.2.3    | The Hadamard Gate and the Diagonal Basis . . . . .                  | 11        |
| 2.3      | The No-Cloning Theorem . . . . .                                    | 11        |
| 2.4      | Heisenberg's Uncertainty Principle in QKD . . . . .                 | 12        |
| 2.5      | The BB84 Protocol . . . . .                                         | 12        |
| 2.6      | Quantum Bit Error Rate (QBER) . . . . .                             | 14        |
| 2.7      | Error Correction and Privacy Amplification . . . . .                | 14        |
| 2.7.1    | Error Correction . . . . .                                          | 14        |
| 2.7.2    | Privacy Amplification . . . . .                                     | 15        |
| 2.8      | Optical-Fibre Channel Attenuation . . . . .                         | 16        |
| 2.9      | Decoy-State Method and the Photon-Number-Splitting Attack . . . . . | 16        |
| 2.10     | The CASCADE Information-Reconciliation Algorithm . . . . .          | 18        |
| 2.11     | Machine Learning for QKD Security Monitoring . . . . .              | 19        |
| <b>3</b> | <b>Existing QKD Simulators</b>                                      | <b>20</b> |
| 3.1      | QKDSim . . . . .                                                    | 20        |
| 3.2      | NetSquid . . . . .                                                  | 20        |
| 3.3      | QuNetSim / SimulaQron . . . . .                                     | 20        |
| 3.4      | Positioning of Sequire . . . . .                                    | 20        |
| <b>4</b> | <b>Technologies Used</b>                                            | <b>21</b> |
| 4.1      | Python and Qiskit . . . . .                                         | 21        |
| 4.2      | Qiskit-Aer (Noisy Simulation Backend) . . . . .                     | 21        |
| 4.3      | Qiskit-IBM-Runtime (Real Hardware) . . . . .                        | 21        |
| 4.4      | FastAPI . . . . .                                                   | 21        |
| 4.5      | WebSockets for Real-Time Streaming . . . . .                        | 21        |
| 4.6      | SQLite for Run Persistence . . . . .                                | 21        |
| 4.7      | React with Vite . . . . .                                           | 21        |
| 4.8      | Tailwind CSS . . . . .                                              | 21        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>5</b> | <b>Application Architecture</b>                       | <b>22</b> |
| 5.1      | The C4 Model . . . . .                                | 22        |
| 5.1.1    | Level 1: System Context . . . . .                     | 22        |
| 5.1.2    | Level 2: Containers . . . . .                         | 22        |
| 5.1.3    | Level 3: Components . . . . .                         | 22        |
| 5.1.4    | Level 4: Code . . . . .                               | 22        |
| 5.2      | Database Schema . . . . .                             | 22        |
| <b>6</b> | <b>Implementation and Features</b>                    | <b>23</b> |
| 6.1      | User Flow Diagram . . . . .                           | 23        |
| 6.2      | Configuration Panel . . . . .                         | 23        |
| 6.3      | The BB84 Protocol Engine . . . . .                    | 23        |
| 6.4      | Eve Eavesdropping Simulation . . . . .                | 23        |
| 6.5      | IBM Quantum Hardware Integration . . . . .            | 23        |
| 6.6      | Real-Time WebSocket Streaming . . . . .               | 23        |
| 6.7      | PhotonGrid Visualization . . . . .                    | 23        |
| 6.8      | Key Distillation Pipeline . . . . .                   | 23        |
| 6.9      | QBER Historical Chart . . . . .                       | 23        |
| 6.10     | Educational Panel . . . . .                           | 23        |
| 6.11     | Simulation History and Persistence . . . . .          | 23        |
| <b>7</b> | <b>Experimental Results</b>                           | <b>24</b> |
| 7.1      | Methodology . . . . .                                 | 24        |
| 7.2      | Simulator vs. Real IBM Hardware . . . . .             | 24        |
| 7.3      | QBER Distribution under Different Eve Modes . . . . . | 24        |
| 7.4      | Impact of Noise Parameters on Key Yield . . . . .     | 24        |
| <b>8</b> | <b>Conclusions</b>                                    | <b>25</b> |
| 8.1      | Future Development Possibilities . . . . .            | 25        |
| 8.2      | General Achievements and Conclusions . . . . .        | 25        |

# 1 Introduction

## 1.1 Motivation

The security of virtually all digital communication today rests on the computational difficulty of certain mathematical problems — factoring large integers in the case of RSA, and computing discrete logarithms in the case of elliptic-curve cryptography (ECC). These schemes are not proven secure in an information-theoretic sense; they are secure only because no classical polynomial-time algorithm is known to break them.

This foundation was shaken in 1994, when Peter Shor demonstrated that a sufficiently large quantum computer could factor an  $n$ -bit integer in polynomial time [2]. Although practical fault-tolerant quantum computers capable of breaking 2048-bit RSA remain several years away, the cryptographic community has already entered a transition period. The *harvest now, decrypt later* strategy — in which adversaries collect encrypted traffic today with the intent to decrypt it once quantum hardware matures — represents a concrete, near-term threat to the confidentiality of long-lived sensitive data [3].

In response to this threat, the National Institute of Standards and Technology (NIST) launched a post-quantum cryptography standardisation process in 2016, selecting its first set of quantum-resistant algorithms in 2024. However, *post-quantum cryptography* (PQC) still relies on mathematical hardness assumptions, merely choosing problems believed to be resistant to quantum algorithms. A fundamentally different approach, one that achieves *information-theoretic* security whose guarantees are grounded in the laws of physics rather than computational assumptions, is offered by **Quantum Key Distribution** (QKD).

QKD allows two parties, commonly referred to as Alice and Bob, to establish a shared secret key over an insecure channel such that any eavesdropping attempt by a third party (Eve) is detectable with overwhelming probability. The security follows from two foundational principles of quantum mechanics: the **no-cloning theorem**, which forbids copying an arbitrary unknown quantum state [4], and the disturbance introduced by measurement in an incompatible basis, as formalised by the **Heisenberg uncertainty principle**. Any interception by Eve necessarily disturbs the quantum states she measures, leaving a statistically detectable signature in the error rate of the received key.

The first and most influential QKD protocol was proposed by Charles Bennett and Gilles Brassard in 1984 [1]. The BB84 protocol encodes classical bits into the polarisation states of individual photons using two mutually unbiased bases, and its unconditional security has been rigorously proved under realistic noise models [5, 6]. Despite its theoretical

maturity, however, BB84 remains inaccessible to most students and researchers for practical experimentation: real QKD hardware is prohibitively expensive, and existing software simulators are either too narrowly focused on performance benchmarking, lack real-time visualisation, or do not integrate with actual quantum processors.

This accessibility gap motivates the present work. There is a clear need for an interactive, web-based platform that can demonstrate the full BB84 pipeline — from qubit generation through to a derived 128-bit secret key — while offering real hardware execution on IBM Quantum backends and providing a step-by-step visual intuition for every stage of the protocol. Such a tool would serve both an educational purpose, making quantum cryptography tangible for students encountering it for the first time, and a research purpose, providing a configurable testbed for studying protocol behaviour under varied noise regimes and adversarial conditions.

## 1.2 Proposed Solution

**Sequire** is a real-time simulation and visualisation platform for the BB84 Quantum Key Distribution protocol. It is implemented as a web application accessible through any modern browser, with no local installation required beyond a running Docker environment. The platform provides a complete, end-to-end demonstration of the quantum cryptographic pipeline under three distinct execution modes.

At its core, Sequire implements the full BB84 protocol in software using **Qiskit** [7] for quantum circuit construction and **Qiskit-Aer** for noise-model simulation. Alice encodes classical bits into single-qubit states drawn from the rectilinear basis ( $|0\rangle$ ,  $|1\rangle$ ) or the diagonal basis ( $|+\rangle$ ,  $|-\rangle$ ), and Bob measures each received qubit in a randomly chosen basis. Following the exchange, the sifting step retains only those bits where Alice and Bob chose the same basis (on average 50% of transmitted qubits). The platform then computes the **Quantum Bit Error Rate** (QBER), applies error correction via a parity-block reconciliation scheme, and performs privacy amplification by truncating a SHA-256 digest of the reconciled key to 128 bits.

The simulation operates in one of three modes selectable by the user:

- **Ideal simulation:** noise-free quantum circuits on a statevector or QASM simulator, producing a zero-QBER baseline.
- **Noisy simulation:** configurable depolarising and measurement errors applied via Qiskit-Aer, producing realistic QBER distributions that allow the user to study the protocol’s behaviour as a function of hardware imperfection.
- **Real IBM Quantum hardware:** circuits are transpiled and dispatched to a real superconducting quantum processor via **Qiskit IBM Runtime** (SamplerV2 API),

currently supporting the `ibm_fez`, `ibm_marrakesh`, and `ibm_kingston` backends. Results are polled asynchronously and streamed live to the frontend.

Beyond the core protocol, Sequire implements several layers of physical and analytical realism that bring the simulator closer to a research-grade testbed:

- **Optical-fibre channel attenuation.** Each transmitted photon survives the channel with probability  $\eta(L) = 10^{-\alpha L/10}$ , where  $\alpha = 0.2$  dB/km is the standard single-mode-fibre loss coefficient and  $L$  is the configurable transmission distance. A dedicated chart plots the asymptotic secure key rate  $R \sim \eta(L)(1 - 2h(Q))$  across 0–200 km, illustrating the well-known rate–distance limits of point-to-point QKD without trusted nodes.
- **Decoy-state BB84 with weak coherent pulses.** The user can switch from an idealised single-photon source to a *weak-coherent-pulse* (WCP) source emitting two intensity classes (signal  $\mu_s$  and decoy  $\mu_d$ ), with Poissonian photon-number statistics. Per-intensity gains and error rates are tracked online, and the GLLP two-decoy bounds are evaluated to estimate the single-photon yield  $Y_1$  and error rate  $e_1$  — the quantities required for security against the photon-number-splitting (PNS) attack.
- **Full CASCADE error correction.** The original parity-block scheme was replaced with the multi-pass CASCADE algorithm with back-correction, exposing both the corrected key and the information leakage  $\ell$  that is later consumed by privacy amplification.
- **Machine-learning eavesdropper detection.** A Random Forest classifier trained on six contextual features per run (QBER, sifting efficiency, depolarising noise, measurement-error rate, channel distance, protocol length) produces an estimated probability that an eavesdropper was present. The classifier is retrainable on demand from the SQLite simulation history and provides a security signal that operates orthogonally to the QBER threshold.
- **User accounts and per-user attribution of runs.** A full authentication layer (email/password with bcrypt, JWT-based sessions in HttpOnly cookies, CSRF double-submit protection, password reset, and Google OAuth 2.0 sign-in) allows simulation runs to be attributed to authenticated users while keeping anonymous access fully functional.

An optional **Eve eavesdropping simulation** implements three distinct attack strategies of progressively increasing sophistication: *Weak Eve*, which intercepts 30% of transmitted qubits; *Strong Eve*, which intercepts all qubits and reliably generates a QBER of approximately 25%, well above the 11% security threshold; and *Smart Eve*, an adaptive intercept-resend adversary that uses a proportional feedback loop on the

running QBER to tune its interception probability towards a user-configurable target (e.g. 9%, just below the abort threshold), thereby evading a naive QBER-only security check. The Smart Eve mode motivates the machine-learning detector described above and provides a concrete adversarial baseline against which the classifier is evaluated.

The user interface, built with **React** (Vite) and **Tailwind CSS**, receives per-qubit events from the backend over a persistent WebSocket connection and renders them live without page refresh. The **PhotonGrid** component animates the first 80 qubit exchanges cell by cell, encoding each cell’s basis symbol, bit value, and Bob’s measurement basis, and highlights Eve-intercepted qubits with a visual marker. A **key distillation pipeline** component visualises the progressive shrinkage of the key through each post-processing stage, while the **QBER historical chart** plots error rates across all previous simulation runs against the security threshold line. An **educational panel** of five expandable cards introduces the underlying theory in accessible language alongside the simulation results.

Completed simulation runs are automatically persisted to a **SQLite** database, and REST endpoints allow history retrieval and selective deletion, enabling longitudinal analysis of QBER behaviour across multiple experiments.

### 1.3 Contributions and Impact

The primary contributions of this work are the following:

- C1. A fully functional, end-to-end BB84 implementation on real quantum hardware.** Unlike purely classical simulators, Sequire executes single-qubit BB84 circuits on IBM superconducting processors via the Qiskit IBM Runtime SamplerV2 interface. This integration enables the direct observation of real decoherence effects — gate errors, readout noise, crosstalk — on the QKD error rate, a dimension that no software-only simulator can replicate.
- C2. Real-time, event-driven visualisation of the quantum protocol.** The WebSocket streaming architecture delivers per-qubit simulation events to the frontend as they are generated, producing a live, animated view of the key exchange. This level of temporal granularity is absent from existing QKD educational tools, which typically present only aggregate results after a simulation completes.
- C3. A spectrum of eavesdropping models, from passive to adaptive.** Beyond the textbook intercept-resend Eve, Sequire implements a *Smart Eve* adversary that uses a proportional feedback loop on the running QBER to evade the standard abort threshold while still extracting partial information. Combined with the QBER-based security check, this stack of three Eve modes (Weak, Strong, Smart)

demonstrates concretely both the security guarantees and the *limitations* of a threshold-only check against an intelligent adversary.

- C4. A realistic optical-fibre channel model with rate–distance curves.** A configurable attenuation coefficient ( $\alpha = 0.2 \text{ dB/km}$  by default) is applied per qubit, and the asymptotic secure key rate is plotted as a function of channel length, bringing the simulator in line with standard QKD benchmarking methodology.
- C5. Decoy-state BB84 with weak-coherent-pulse modelling.** Two intensity classes (signal and decoy) are emitted with Poissonian photon-number statistics, and the GLLP two-decoy bounds are evaluated online to recover the single-photon yield and error rate — the security-critical quantities under the PNS attack.
- C6. Full CASCADE error correction with information-leakage accounting.** The multi-pass binary-search reconciliation with back-correction replaces the lossy parity-block scheme and exposes the leakage  $\ell$  that feeds into privacy amplification, closing the loop on a rigorous post-processing pipeline.
- C7. Supervised eavesdropper detection via machine learning.** A Random Forest classifier trained on six contextual features per run produces a probability of eavesdropper presence that is complementary to (and orthogonal in failure mode to) the QBER threshold, providing a defence against precisely the adaptive adversaries that the QBER check cannot catch on its own.
- C8. Multi-user platform with authenticated session management.** A full authentication layer (email/password with bcrypt, JWT access/refresh tokens in HttpOnly cookies, CSRF double-submit protection, password reset, Google OAuth 2.0) allows simulations to be attributed to authenticated users, paving the way for the upcoming gamified and multi-player modes while preserving anonymous guest access.
- C9. A configurable research testbed for QKD experimentation.** The platform exposes noise parameters (depolarising probability, measurement error rate, interception fraction, channel distance, source type, decoy intensities, error-correction algorithm) as user-controlled inputs, making it suitable not only for classroom demonstration but also as an exploratory tool for studying the protocol's behaviour under varied hardware and adversarial conditions.
- C10. Persistent simulation history with REST API access.** Automated SQLite persistence of every simulation run, queryable via REST endpoints, enables longitudinal analysis and reproducible comparison of results across hardware modes,

noise levels, and Eve configurations, while also serving as the training corpus for the ML classifier.

Beyond its immediate academic scope, Sequire is conceived as a research-oriented testbed with a long-term ambition: to probe the practical limits of quantum key distribution under realistic hardware imperfections, advanced attack models, and extended protocol variants. The architecture is deliberately modular — the quantum logic, classical post-processing, and presentation layers are cleanly separated — and this modularity has already enabled the integration of decoy-state BB84, CASCADE error correction, an optical-fibre attenuation model, an adaptive (Smart) Eve, and an ML-based detection layer into the same code base without restructuring. Further extensions identified for the present cycle and discussed in Section 8 include alternative QKD protocols (E91, B92, SARG04), an explicit photon-number-splitting attack, finite-key security bounds, and a single-player gamified mode in which the user plays the role of Eve and is scored against the ML detector.

## 2 Theoretical Background

This chapter provides the theoretical foundations necessary to understand the Sequire platform and the BB84 protocol it implements. It begins with a comparison of classical and quantum approaches to cryptographic key distribution, introduces the relevant quantum mechanical formalism, and then describes in detail the BB84 protocol, its security metric (QBER), and the classical post-processing steps that transform raw qubit measurements into a usable secret key.

### 2.1 Classical vs. Quantum Cryptography

Classical cryptography can be divided into two broad categories: *symmetric-key* cryptography, in which sender and receiver share a common secret key, and *public-key* (asymmetric) cryptography, in which a public key is used for encryption and a private key for decryption. The security of public-key schemes such as RSA and elliptic-curve cryptography (ECC) relies on the computational intractability of certain mathematical problems — integer factorisation and the discrete logarithm problem, respectively. These problems are hard for classical computers, but as established in Section 1.1, Shor's algorithm [2] solves both in polynomial time on a sufficiently powerful quantum computer. Symmetric schemes are weakened but not broken: Grover's algorithm reduces an  $n$ -bit key search from  $O(2^n)$  to  $O(2^{n/2})$  [8], so doubling the key length restores the original security margin.

**Quantum Key Distribution** takes an entirely different approach. Rather than relying on mathematical assumptions, QKD uses the physical properties of quantum states to distribute a key in such a way that any interception attempt is detectable in principle. The security is *information-theoretic*: it holds regardless of the computational power of the adversary. The key insight is that quantum mechanics imposes a fundamental constraint on any measurement process — measuring a quantum state in an incompatible basis irreversibly disturbs that state, leaving a detectable trace.

Table 1 summarises the key differences between the two paradigms.

Table 1 – Classical vs. quantum key distribution — a comparison.

| Property            | Classical (RSA/ECC)        | QKD (BB84)                                 |
|---------------------|----------------------------|--------------------------------------------|
| Security basis      | Computational hardness     | Laws of quantum physics                    |
| Quantum threat      | Broken by Shor's algorithm | Unaffected                                 |
| Eavesdrop detection | Not inherent               | Guaranteed by physics                      |
| Key distribution    | Public-key exchange        | Quantum channel + public classical channel |
| Security model      | Computational              | Information-theoretic                      |

## 2.2 Quantum Mechanics Primer

### 2.2.1 Qubits and Quantum States

The fundamental unit of quantum information is the **qubit** (quantum bit). Unlike a classical bit, which is either 0 or 1, a qubit can exist in a *superposition* of both basis states simultaneously. Using Dirac (bra-ket) notation, the general state of a qubit is written as:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (1)$$

where  $|0\rangle$  and  $|1\rangle$  are the computational basis states (eigenstates of the Pauli- $Z$  operator), and  $|\alpha|^2$ ,  $|\beta|^2$  are the probabilities of measuring the qubit in state  $|0\rangle$  or  $|1\rangle$ , respectively.

### 2.2.2 Measurement and State Collapse

A key property of quantum mechanics is that measuring a qubit in a given basis *collapses* its state to one of the basis vectors. If the qubit is in state  $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$  and we measure in the  $Z$ -basis  $\{|0\rangle, |1\rangle\}$ , we obtain outcome 0 with probability  $|\alpha|^2$  and outcome 1 with probability  $|\beta|^2$ , and the state collapses accordingly. After the measurement, all information about the pre-measurement superposition is irrevocably lost.

Crucially, if a qubit is prepared in a state belonging to one orthonormal basis and then measured in a *different*, incompatible basis, the outcome is uniformly random and the original state cannot be inferred. This is the physical mechanism exploited by BB84 to detect eavesdropping.

### 2.2.3 The Hadamard Gate and the Diagonal Basis

In addition to the computational  $Z$ -basis  $\{|0\rangle, |1\rangle\}$ , BB84 uses the *diagonal* ( $X$ -basis)  $\{|+\rangle, |-\rangle\}$ , defined by:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \quad (2)$$

The transformation between the two bases is performed by the **Hadamard gate**  $H$ :

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle. \quad (3)$$

In the Sequire implementation, the Hadamard gate is applied in Qiskit via `qc.h(0)` when Alice encodes a bit in the  $X$ -basis, and equally when Bob measures in the  $X$ -basis (rotating back before the standard  $Z$ -measurement).

## 2.3 The No-Cloning Theorem

**Theorem** (Wootters and Zurek, 1982 [4]): There is no unitary operation  $U$  that can copy an arbitrary unknown quantum state. Formally, there exists no  $U$  such that  $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$  for all  $|\psi\rangle$ .

*Proof sketch.* Suppose such a  $U$  existed. Then for any two states  $|\psi\rangle$  and  $|\phi\rangle$ :

$$U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle, \quad (4)$$

$$U(|\phi\rangle|0\rangle) = |\phi\rangle|\phi\rangle. \quad (5)$$

Taking the inner product of both sides:

$$\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2. \quad (6)$$

This equation holds only if  $\langle\psi|\phi\rangle = 0$  (orthogonal states) or  $\langle\psi|\phi\rangle = 1$  (identical states). For any other pair of states,  $0 < |\langle\psi|\phi\rangle| < 1$ , so no such  $U$  can exist.  $\square$

**Implication for QKD.** The no-cloning theorem makes it impossible for Eve to intercept a transmitted qubit, make a perfect copy, forward the original to Bob, and measure the copy at her leisure. Any attempt by Eve to gain information about the qubit requires her to interact with it, which — as the following section explains — necessarily disturbs the quantum state.

## 2.4 Heisenberg's Uncertainty Principle in QKD

Heisenberg's uncertainty principle states that certain pairs of physical properties, known as *complementary observables*, cannot both be known simultaneously with arbitrary precision. In the context of BB84, the relevant complementary observables are the Pauli- $Z$  and Pauli- $X$  operators, corresponding to measurement in the computational basis and the diagonal basis, respectively.

Consider a qubit sent by Alice in the  $Z$ -basis in state  $|0\rangle$ . If Bob — or Eve — measures this qubit in the  $X$ -basis instead, the outcome is uniformly random:  $|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$ , so the result is  $+$  or  $-$  with equal probability  $\frac{1}{2}$ . Moreover, after this  $X$ -basis measurement, the qubit's state has collapsed to either  $|+\rangle$  or  $|-\rangle$  — no longer  $|0\rangle$ . If this disturbed qubit is forwarded to Bob and he measures in the  $Z$ -basis (as Alice intended), his result is again uniformly random, with a 50% probability of disagreeing with Alice's original bit.

This is the precise mechanism by which eavesdropping is detected in BB84:

- When Eve intercepts a qubit, she must choose a measurement basis without knowing which basis Alice used.
- She guesses the correct basis with probability  $\frac{1}{2}$ .
- When she guesses wrong (50% of intercepts), her measurement disturbs the state.
- Of those disturbed qubits, Bob's measurement disagrees with Alice's original bit with probability  $\frac{1}{2}$ .

Each intercepted qubit therefore introduces an error with probability  $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$ . A complete intercept-resend attack (Strong Eve) thus raises the QBER to approximately 25%.

## 2.5 The BB84 Protocol

The BB84 protocol [1] proceeds as follows.

**Step 1 — Qubit preparation (Alice).** Alice generates a random bit string  $\mathbf{a} = (a_1, \dots, a_n)$  and an independent random basis string  $\mathbf{x} = (x_1, \dots, x_n)$ , where each  $x_i \in \{Z, X\}$ . She encodes each bit  $a_i$  into a qubit according to Table 2 and transmits the qubits one by one over the quantum channel.

Table 2 – BB84 encoding: the four qubit states used in the protocol.

| Bit | Basis           | State                                                   | Description                       |
|-----|-----------------|---------------------------------------------------------|-----------------------------------|
| 0   | Z (rectilinear) | $ 0\rangle$                                             | spin-up / horizontal polarisation |
| 1   | Z (rectilinear) | $ 1\rangle$                                             | spin-down / vertical polarisation |
| 0   | X (diagonal)    | $ +\rangle = \frac{1}{\sqrt{2}}( 0\rangle +  1\rangle)$ | +45° polarisation                 |
| 1   | X (diagonal)    | $ -\rangle = \frac{1}{\sqrt{2}}( 0\rangle -  1\rangle)$ | -45° polarisation                 |

**Step 2 — Qubit measurement (Bob).** Bob independently chooses a random basis string  $\mathbf{y} = (y_1, \dots, y_n)$  and measures each received qubit in the corresponding basis, recording his outcomes  $\mathbf{b} = (b_1, \dots, b_n)$ .

**Step 3 — Basis sifting.** Alice and Bob communicate their basis choices  $\mathbf{x}$  and  $\mathbf{y}$  over a public classical channel (no bit values are revealed). They retain only the positions where  $x_i = y_i$  — on average, 50% of all transmitted qubits. The retained bits form the *sifted key*. In Sequire, this step is implemented in `classical_processing/sifting.py`:

```

1 def sift_keys(alice_bits, alice_bases, bob_bits, bob_bases):
2     alice_key, bob_key = [], []
3     for a_bit, a_basis, b_bit, b_basis in zip(
4         alice_bits, alice_bases, bob_bits, bob_bases):
5         if a_basis == b_basis:
6             alice_key.append(a_bit)
7             bob_key.append(b_bit)
8     return alice_key, bob_key

```

Listing 1: Basis sifting in Sequire.

**Step 4 — Error rate estimation (QBER).** Alice and Bob sacrifice a random subset of their sifted key to estimate the Quantum Bit Error Rate (QBER). If QBER exceeds a security threshold, they abort the protocol (see Section 2.6).

**Step 5 — Error correction.** Alice and Bob apply an error-correction protocol over the public channel to reconcile the remaining bits of their sifted keys (see Section 2.7).

**Step 6 — Privacy amplification.** To account for any partial information Eve may have obtained, Alice and Bob apply a universal hash function to compress their reconciled key into a shorter but statistically independent final key (see Section 2.7).

## 2.6 Quantum Bit Error Rate (QBER)

The **Quantum Bit Error Rate** (QBER) is defined as the fraction of sifted key bits on which Alice and Bob disagree:

$$Q = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[a_i \neq b_i], \quad (7)$$

where  $N$  is the length of the sifted key and  $a_i, b_i$  are the corresponding bits held by Alice and Bob after sifting.

In a noise-free channel without eavesdropping,  $Q = 0$ . Channel noise (depolarising errors, measurement errors) raises  $Q$  by a small, bounded amount. An active intercept-resend attack raises  $Q$  significantly above any noise floor. Specifically:

- **No Eve, no noise:**  $Q \approx 0$ .
- **Weak Eve** (intercepts fraction  $p$  of qubits): each intercepted qubit introduces an error with probability  $\frac{1}{4}$  (wrong basis with probability  $\frac{1}{2}$ , error given wrong basis with probability  $\frac{1}{2}$ ), so  $Q \approx \frac{p}{4}$ . For Sequire's Weak Eve ( $p = 0.30$ ), this gives  $Q \approx 7.5\%$ .
- **Strong Eve** ( $p = 1.0$ , full intercept-resend):  $Q \approx 25\%$ .

Sequire uses a security threshold of  $Q_{\text{th}} = 11\%$ , consistent with the standard BB84 security analysis [9]: if  $Q \geq 11\%$ , the protocol is aborted and no key is output. This threshold ensures that any remaining information Eve holds about the key can be made exponentially small by privacy amplification.

## 2.7 Error Correction and Privacy Amplification

### 2.7.1 Error Correction

Even in the absence of eavesdropping, channel noise causes a non-zero QBER. Before a key can be used, Alice and Bob must reconcile their sifted keys so that they hold identical bit strings. This is performed via an *information reconciliation* protocol over the public classical channel.

Sequire implements a simplified parity-block scheme: the sifted key is divided into consecutive blocks of  $B = 4$  bits. For each block, Alice announces the block's parity (XOR of its bits) over the public channel. Bob computes the parity of his corresponding block. If the parities agree, both parties retain the block. If they disagree (indicating at least one error within the block), the entire block is discarded by both parties. The implementation in `classical_processing/error_correction.py` is:

```
1 def correct_errors(alice_key, bob_key, block_size=4):
```

```

2   corrected_alice, corrected_bob = [], []
3   for i in range(0, len(alice_key) - block_size + 1, block_size):
4       a_block = alice_key[i:i + block_size]
5       b_block = bob_key[i:i + block_size]
6       if parity(a_block) == parity(b_block):
7           corrected_alice.extend(a_block)
8           corrected_bob.extend(b_block)
9   return corrected_alice, corrected_bob

```

Listing 2: Parity-block error correction in Sequire.

This approach sacrifices key material (entire blocks are discarded rather than individual erroneous bits), but guarantees that all retained blocks are error-free with high probability for low to moderate QBER. The full **CASCADE** algorithm [11] improves upon this by iteratively narrowing down the position of individual erroneous bits through binary search, achieving near-optimal reconciliation efficiency. CASCADE is identified as a future development direction for Sequire (see Section 8).

### 2.7.2 Privacy Amplification

After error correction, Alice and Bob share an identical key, but Eve may hold partial information about it: she observed parity announcements over the public channel during error correction, and she may have obtained information from any qubits she intercepted before the QBER check. Privacy amplification *compresses* the reconciled key using a hash function, reducing Eve's information about the output key to a negligible quantity regardless of how much she learned about the input [10].

Sequire implements privacy amplification by computing the **SHA-256** hash of the reconciled key and truncating the digest to 128 bits (32 hexadecimal characters):

```

1   def amplify_privacy(key, output_bits=128):
2       byte_length = (len(key) + 7) // 8
3       key_int = int("".join(str(b) for b in key), 2) if key else 0
4       key_bytes = key_int.to_bytes(byte_length, byteorder="big")
5       digest = hashlib.sha256(key_bytes).hexdigest()
6       return digest[: output_bits // 4] # 32 hex chars = 128 bits

```

Listing 3: SHA-256 privacy amplification in Sequire.

SHA-256 is a collision-resistant, one-way function and serves as a practical approximation to the universal hash families used in the information-theoretic proofs of privacy amplification. While not strictly optimal from a universal-hashing standpoint — optimal constructions use seeded hash families whose seed is shared publicly — SHA-256 truncation is a common and accepted choice for prototype implementations, and provides 128 bits of output security in the absence of attacks on the hash function itself.

## 2.8 Optical-Fibre Channel Attenuation

The discussion so far has treated the quantum channel as a passive conduit that may add gate-level noise but transmits every qubit. In any physical implementation of QKD, however, the dominant cause of bit loss is not noise but *photon attenuation* along the channel. For optical fibre links, the survival probability of a photon transmitted over a distance  $L$  (in km) follows the Beer–Lambert law:

$$\eta(L) = 10^{-\alpha L/10}, \quad (8)$$

where  $\alpha$  is the fibre's loss coefficient. The de-facto industry value for telecom-grade single-mode fibre at the 1550 nm wavelength is  $\alpha \approx 0.2$  dB/km [12]. This implies, for instance, that only  $\eta(50) \approx 10\%$  of photons survive a 50-km link and  $\eta(100) \approx 1\%$  survive 100 km. Free-space links incur instead *beam-divergence* loss, modelled as  $\eta \propto 1/L^2$  but with substantially smaller coefficients per unit distance over short ranges.

The practical consequence for QKD is that the *secure key rate* — the number of secret-key bits produced per channel use — decays exponentially with distance. In the asymptotic (large- $N$ ) regime, the secret-key rate for BB84 with one-way classical communication is given by the GLLP–Shor–Preskill formula [9, 13]:

$$R(L) \geq \eta(L) \left[ 1 - H_2(e_b) - f(e_b) H_2(e_b) \right], \quad (9)$$

where  $e_b$  is the bit-error rate,  $H_2$  is the binary entropy  $H_2(x) = -x \log_2 x - (1-x) \log_2 (1-x)$ , and  $f(e_b) \geq 1$  is the efficiency factor of the error-correction algorithm relative to the Shannon limit. The term  $1 - H_2(e_b)$  accounts for the information Eve could have gained from the quantum channel; the term  $f(e_b) H_2(e_b)$  discounts the bits revealed during classical reconciliation.

Equation (9) sets the well-known *rate–distance trade-off* of BB84: there exists a maximum distance  $L^*$  beyond which the right-hand side becomes negative and no positive key rate can be obtained. For  $\alpha = 0.2$  dB/km,  $e_b = 1\%$ , and CASCADE-typical  $f \approx 1.16$ ,  $L^* \approx 150$ –200 km. Surpassing this limit requires either trusted intermediate nodes, twin-field protocols [14], or quantum repeaters — a key observation that motivates the Sequire rate-vs-distance visualisation in Section 6.

## 2.9 Decoy-State Method and the Photon-Number-Splitting Attack

The BB84 protocol as originally formulated assumes Alice transmits a *single-photon source* (SPS), so that any attempt by Eve to extract information necessarily perturbs

the qubit. In practice, true single-photon sources are demanding to engineer; instead, the overwhelming majority of deployed QKD systems use attenuated lasers — so-called *weak coherent pulses* (WCP). A WCP with mean photon number  $\mu$  emits, per pulse,  $n$  photons with probability:

$$P(n | \mu) = e^{-\mu} \frac{\mu^n}{n!}. \quad (10)$$

For  $\mu < 1$ , the dominant contributions are  $n = 0$  (vacuum) and  $n = 1$  (single photon), but a non-negligible fraction of pulses contains  $n \geq 2$  photons. These multi-photon pulses expose the protocol to a **photon-number-splitting** (PNS) attack: Eve installs a quantum non-demolition photon-number measurement at the output of Alice, blocks all  $n = 1$  pulses, and for every  $n \geq 2$  pulse keeps one photon in a quantum memory while forwarding the rest to Bob unaltered. After Alice and Bob publicly announce their bases, Eve measures her stored photons in the correct basis, obtaining the bit value with no QBER penalty whatsoever.

The PNS attack is fatal to naive WCP-BB84 over high-loss channels, because in the limit  $\eta(L) \rightarrow 0$  the legitimate single-photon signal can be completely suppressed in favour of intercepted multi-photon contributions without raising the QBER above the security threshold. The **decoy-state method** [15, 16] elegantly defeats this attack by having Alice randomly switch among several intensity classes:

- a *signal* intensity  $\mu_s$  used for key generation;
- one or more *decoy* intensities  $\mu_d, \nu, \dots$  used only for monitoring.

Because Eve does not know in advance the intensity of any individual pulse, she cannot distinguish signal multi-photon pulses from decoy multi-photon pulses. The per-intensity gains  $Q_\mu = \sum_n P(n | \mu) Y_n$  and error rates  $E_\mu$  provide a system of linear inequalities that bound the *single-photon yield*  $Y_1$  and *single-photon error rate*  $e_1$  in a verifiable way. In the two-decoy GLLP analysis, the lower bound on  $Y_1$  is:

$$Y_1^L \geq \frac{\mu}{\mu\nu - \nu^2} \left( Q_\nu e^\nu - Q_\mu e^\mu \frac{\nu^2}{\mu^2} - \frac{\mu^2 - \nu^2}{\mu^2} Y_0 \right), \quad (11)$$

where  $\mu$  and  $\nu$  are the signal and decoy means and  $Y_0$  is the background (dark-count) rate. The secure key rate is then:

$$R \geq q \left[ -Q_\mu f(E_\mu) H_2(E_\mu) + Q_1^L (1 - H_2(e_1^U)) \right], \quad (12)$$

with  $Q_1^L = \mu e^{-\mu} Y_1^L$ . The Sequire decoy-state panel renders each of these quantities live, so the student can watch the single-photon contribution to the key being recovered from the otherwise aggregate WCP statistics.

## 2.10 The CASCADE Information-Reconciliation Algorithm

The block-parity reconciliation described in Section 2.7 is a useful pedagogical baseline but is highly suboptimal: it discards entire blocks on any disagreement, leaking only  $\log_2(B)$  bits less than the optimum per corrected error but losing  $B$  bits of key in the process. The CASCADE algorithm [11], introduced by Brassard and Salvail in 1993, remains the de-facto reference for QKD error correction to this day because it isolates and corrects individual erroneous bits rather than discarding their surrounding blocks.

CASCADE proceeds in  $k$  *passes*, each pass parameterised by a block size  $B_i$ . The first block size is chosen as a function of the estimated QBER  $Q$ :

$$B_1 \approx \left\lceil \frac{0.73}{Q} \right\rceil, \quad B_{i+1} = 2 B_i. \quad (13)$$

Within a pass, the key is permuted by a fresh random permutation  $\pi_i$  and partitioned into consecutive blocks of size  $B_i$ . Alice announces the parity of each block over the public channel; for every block whose parity differs from Bob’s, the parties perform a binary search — recursively bisecting the block and comparing parities of the halves — until a single erroneous bit is located and flipped.

The crucial element of CASCADE is its **back-correction** mechanism: flipping a bit in pass  $i$  changes the parities of every block in passes  $1, \dots, i-1$  that contained that bit. A flipped bit in a previously “correct” block must have masked a second error. CASCADE therefore revisits all earlier blocks whose parity has now changed and continues binary-searching for the remaining errors. This cross-pass propagation allows the algorithm to converge with high probability after only four to six passes, leaking

$$\ell \approx N \cdot f(Q) \cdot H_2(Q) \quad (14)$$

bits over the public channel, with  $f(Q) \approx 1.05$ – $1.22$  for typical QKD QBER values — close to the Shannon lower bound of  $f = 1$  [17]. The leakage  $\ell$  is precisely the quantity subtracted during privacy amplification to obtain the final secret-key length.

In Sequire, CASCADE is implemented in `backend/classical_processing/cascade.py` and replaces the block-parity scheme. The legacy scheme remains available behind a UI selector for didactic comparison: a side-by-side view shows how many bits are wasted by block discarding versus how many leak through CASCADE parities at the same input QBER.

## 2.11 Machine Learning for QKD Security Monitoring

A QBER-threshold check is a powerful but coarse instrument. It catches any adversary that exceeds the abort threshold but is, by construction, blind to attacks calibrated to stay below it — attacks that Sequire’s Smart Eve mode produces by design. A complementary, statistics-driven detector is therefore desirable.

The detection of subtle adversarial behaviour from an experiment’s contextual signature (QBER, sifting efficiency, noise parameters, distance, sample size) is a textbook *binary classification* problem: given a feature vector  $\mathbf{x} \in \mathbb{R}^d$ , predict the label  $y \in \{0, 1\}$  (*Eve absent*, *Eve present*). Among classical classifiers, the **Random Forest** [18] is particularly well-suited to this setting: it is a non-linear ensemble of decision trees that handles feature interactions natively, is robust to the heterogeneous, non-Gaussian distributions characteristic of QKD features, exposes interpretable feature importances, and trains in seconds on the dataset sizes ( $10^2$ – $10^3$  runs) typical of an educational platform.

Concretely, the model computes

$$\hat{P}(y = 1 \mid \mathbf{x}) = \frac{1}{T} \sum_{t=1}^T \hat{P}_t(y = 1 \mid \mathbf{x}), \quad (15)$$

where each  $\hat{P}_t$  is the leaf-frequency estimate of the  $t$ -th tree trained on a bootstrap sample of the historical runs. The thesis uses  $T = 100$  trees with the scikit-learn default depth limits. Labels are derived automatically from the configured `eve_mode` of each historical run: anything other than `eve_mode == 'none'` is treated as a positive instance.

The literature has begun to explore deeper models for this task recently [19, 20]. Deep recurrent networks (LSTM) can in principle ingest the entire per-qubit error history rather than aggregate features, and have been shown to reach  $> 90\%$  accuracy against adaptive attackers in CV-QKD [19]. The choice of a Random Forest in Sequire is a deliberate trade-off: it provides near-state-of-the-art accuracy on aggregate-feature classification with a fraction of the data, the training, and the interpretability cost of a deep model, while leaving the LSTM extension as an explicit future direction (see Section 8).

## 3 Existing QKD Simulators

### 3.1 QKDSim

### 3.2 NetSquid

### 3.3 QuNetSim / SimulaQron

### 3.4 Positioning of Sequire

## 4 Technologies Used

### 4.1 Python and Qiskit

### 4.2 Qiskit-Aer (Noisy Simulation Backend)

### 4.3 Qiskit-IBM-Runtime (Real Hardware)

### 4.4 FastAPI

### 4.5 WebSockets for Real-Time Streaming

### 4.6 SQLite for Run Persistence

### 4.7 React with Vite

### 4.8 Tailwind CSS

## 5 Application Architecture

### 5.1 The C4 Model

#### 5.1.1 Level 1: System Context

#### 5.1.2 Level 2: Containers

#### 5.1.3 Level 3: Components

#### 5.1.4 Level 4: Code

### 5.2 Database Schema

## 6 Implementation and Features

### 6.1 User Flow Diagram

### 6.2 Configuration Panel

### 6.3 The BB84 Protocol Engine

### 6.4 Eve Eavesdropping Simulation

### 6.5 IBM Quantum Hardware Integration

### 6.6 Real-Time WebSocket Streaming

### 6.7 PhotonGrid Visualization

### 6.8 Key Distillation Pipeline

### 6.9 QBER Historical Chart

### 6.10 Educational Panel

### 6.11 Simulation History and Persistence

## 7 Experimental Results

### 7.1 Methodology

### 7.2 Simulator vs. Real IBM Hardware

### 7.3 QBER Distribution under Different Eve Modes

### 7.4 Impact of Noise Parameters on Key Yield

## 8 Conclusions

### 8.1 Future Development Possibilities

### 8.2 General Achievements and Conclusions

## List of Figures

## References

- [1] C. H. Bennett and G. Brassard, *Quantum Cryptography: Public Key Distribution and Coin Tossing*, Proceedings of the IEEE International Conference on Computers, Systems and Signal Processing, Bangalore, India, pp. 175–179, 1984.
- [2] P. W. Shor, *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*, Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS), pp. 124–134, IEEE, 1994.
- [3] M. Mosca, *Cybersecurity in an Era with Quantum Computers: Will We Be Ready?*, IEEE Security & Privacy, 16(5), pp. 38–41, 2018.
- [4] W. K. Wootters and W. H. Zurek, *A Single Quantum Cannot Be Cloned*, Nature, 299, pp. 802–803, 1982.
- [5] D. Mayers, *Unconditional Security in Quantum Cryptography*, Journal of the ACM, 48(3), pp. 351–406, 2001.
- [6] H.-K. Lo and H. F. Chau, *Unconditional Security of Quantum Key Distribution over Arbitrarily Long Distances*, Science, 283(5410), pp. 2050–2056, 1999.
- [7] Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, <https://qiskit.org>, 2024.
- [8] L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219, 1996.
- [9] P. W. Shor and J. Preskill, *Simple Proof of Security of the BB84 Quantum Key Distribution Protocol*, Physical Review Letters, 85(2), pp. 441–444, 2000.
- [10] C. H. Bennett, G. Brassard, C. Crépeau, and U. M. Maurer, *Generalized Privacy Amplification*, IEEE Transactions on Information Theory, 41(6), pp. 1915–1923, 1995.
- [11] G. Brassard and L. Salvail, *Secret-Key Reconciliation by Public Discussion*, Advances in Cryptology — EUROCRYPT 1993, LNCS 765, pp. 410–423, Springer, 1994.
- [12] International Telecommunication Union, *Characteristics of a Single-Mode Optical Fibre and Cable (ITU-T Recommendation G.652)*, 2016.
- [13] D. Gottesman, H.-K. Lo, N. Lütkenhaus, and J. Preskill, *Security of Quantum Key Distribution with Imperfect Devices*, Quantum Information and Computation, 4(5), pp. 325–360, 2004.

- [14] M. Lucamarini, Z. L. Yuan, J. F. Dynes, and A. J. Shields, *Overcoming the Rate–Distance Limit of Quantum Key Distribution without Quantum Repeaters*, *Nature*, 557(7705), pp. 400–403, 2018.
- [15] H.-K. Lo, X. Ma, and K. Chen, *Decoy State Quantum Key Distribution*, *Physical Review Letters*, 94, 230504, 2005.
- [16] X.-B. Wang, *Beating the Photon-Number-Splitting Attack in Practical Quantum Cryptography*, *Physical Review Letters*, 94, 230503, 2005.
- [17] J. Martínez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martín, *Demystifying the Information Reconciliation Protocol Cascade*, *Quantum Information and Computation*, 15(5&6), pp. 453–477, 2015.
- [18] L. Breiman, *Random Forests*, *Machine Learning*, 45(1), pp. 5–32, 2001.
- [19] Y. Xu et al., *Deep-Learning-Based Continuous Attacks on Continuous-Variable Quantum Key Distribution Protocols*, arXiv:2408.12571, 2024.
- [20] S. Zhao et al., *Deep Reinforcement Learning and Variational Autoencoders for Enhanced Quantum Key Distribution Security*, *Results in Engineering*, 2025.